
Centrale Nantes

Semester 1 — ALPRO Project | (M1 CORO DASSIP)

Participants: Denos KUME — Sena FUKABE

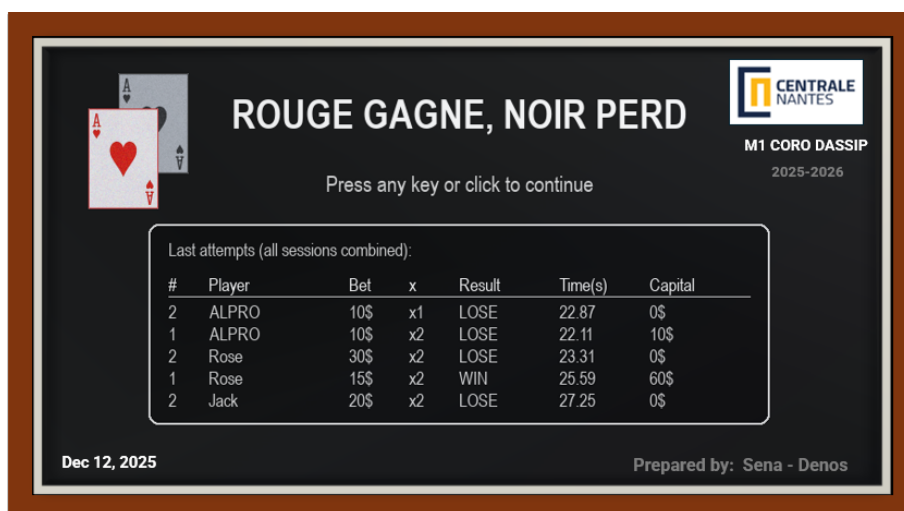
Academic Year 2025-2026 | Instructor: Mira Rizkallah

M1 Control and Robotics – Data Science, Signal and Image Processing

PROJECT REPORT

Rouge Gagne, Noir Perd

Algorithmics and Programming



The screenshot shows a roulette game interface with a dark background and a brown border. In the top left corner, there are three playing cards: the Ace of Hearts, the Ace of Spades, and the Two of Spades. The title 'ROUGE GAGNE, NOIR PERD' is displayed in large white letters. Below the title, it says 'Press any key or click to continue'. In the top right corner, there is a logo for 'CENTRALE NANTES' and text indicating 'M1 CORO DASSIP' and '2025-2026'. A table titled 'Last attempts (all sessions combined):' is shown in the center, listing player performance. At the bottom left, the date 'Dec 12, 2025' is displayed, and at the bottom right, it says 'Prepared by: Sena - Denos'.

#	Player	Bet	x	Result	Time(s)	Capital
2	ALPRO	10\$	x1	LOSE	22.87	0\$
1	ALPRO	10\$	x2	LOSE	22.11	10\$
2	Rose	30\$	x2	LOSE	23.31	0\$
1	Rose	15\$	x2	WIN	25.59	60\$
2	Jack	20\$	x2	LOSE	27.25	0\$

December 12, 2025

ABSTRACT

This document presents the architecture, development methodology, and software engineering principles applied in the interactive card game *Rouge Gagne, Noir Perd*. The system demonstrates practical object-oriented design, state-driven logic, user interface development, and data persistence. This report details the requirements, high-level architecture, class design, SDLC planning, testing, and future recommendations.

ACKNOWLEDGMENT

I would like to express my sincere gratitude to **Mira Rizkallah**, our ALPRO instructor, for her exceptional guidance, dedication, and continuous support throughout this project. Her passion for teaching, her clarity in explaining complex concepts, and her constant encouragement played a pivotal role in shaping both the development of this work and my understanding of algorithmic problem-solving.

Her commitment to student growth and her inspiring approach to computer science have left a lasting impact. This project is proudly dedicated to her as a reflection of the knowledge, discipline, and motivation she instilled in us.

I would also like to express my heartfelt appreciation to my project mate, **Sena FUKABE**, for his dedication, teamwork, and unwavering support throughout the development of this project. His collaborative spirit, thoughtful insights, and consistent reliability greatly enriched the quality of our work and made this experience both productive and enjoyable.

Finally, I would like to extend my sincere thanks to all my classmates for their collaboration, positivity, and shared determination throughout this course. Each of you contributed to an environment where ideas could grow, mistakes became lessons, and challenges turned into opportunities. Your support, hard work, and collective motivation have made this journey truly memorable. I hope we all continue to push forward with the same ambition, confidence, and resilience—carrying these experiences into every future project and achievement ahead.

Sincerely,
Denos KUME

Author

Dec 12, 2025

CONTENTS

1	Introduction	1
1.1	Motivation	1
1.2	Problem Statement	1
2	Requirements Engineering	1
2.1	Functional Requirements	1
2.2	Non-Functional Requirements	2
2.3	System Requirements and Constraints	2
2.4	Glossary of Key Terms	2
3	Development Environment and Libraries	3
3.1	Environment	3
3.2	Libraries Used	3
4	High-Level Design	3
4.1	Project Structure and Modules Overview	3
4.2	UML Class Diagram	5
4.3	ER Diagram	6
4.4	SDLC Gantt Chart	6
4.5	Sequence Diagram	7
5	Object-Oriented Architecture	7
5.1	Encapsulation and Composition	8
5.2	State Machine Logic	8
6	Core Class Implementations	8
6.1	User Class	8
6.2	Bet Class	9
6.3	Card Class	9
6.4	CardGame Class	10

6.5	main function	10
7	User Interface	11
7.1	Main Menu	11
7.2	Bet Dashboard	11
7.3	Step 1: Observe Cards (10s)	12
7.4	Step 2: Cards shuffling (10s)	12
7.5	Step 3: Card Selection (10s)	13
7.6	Result and Game Over Screens	13
8	Software Development Life Cycle	13
9	Risk Analysis	14
9.1	Identified Risks	14
9.2	Mitigation Strategies	14
10	Test Plan and Validation	15
10.1	Functional Testing	15
10.2	Non-Functional Testing	15
11	Summary	16
12	Conclusion and Perspectives	16
A	Source Code Listings	17

LIST OF FIGURES

1	UML Class Diagram.	5
2	Entity–Relationship Diagram for PLAYER and ROUND.	6
3	SDLC Gantt Chart (Design, Implementation, Testing, Documentation). .	6
4	Sequence Diagram: a complete round.	7
5	Main menu: nickname and avatar selection.	11
6	Bet configuration dashboard.	11
7	Result and Game Over dashboard.	13

1 INTRODUCTION

Rouge Gagne, Noir Perd is an interactive card-tracking game built with Python and Pygame. The player must track a red card during a fast shuffle. The system integrates a betting engine, real-time animations, persistent data logging, and a performance dashboard.

1.1 Motivation

The project aims to:

- illustrate event-driven programming in a playful, visual context
- provide a compact but complete example of an object-oriented game
- connect theoretical SDLC and UML concepts with an implemented system
- offer a basis for future extensions such as AI, analytics and multiplayer.

1.2 Problem Statement

The goal is to design and implement a small but realistic software system that:

- guides the player through a clear sequence of states (design, betting, shuffling, selection, result)
- ensures consistent and fair betting logic
- persists game rounds for later inspection
- remains maintainable and extensible for further teaching and experimentation.

2 REQUIREMENTS ENGINEERING

2.1 Functional Requirements

- user profile creation (nickname, avatar)
- bet configuration with min/max enforcement and turbo multiplier
- display of cards, back view, shuffle phase, and timed selection
- win/loss resolution and balance updates
- automatic Game Over when balance is insufficient
- JSON persistence of each round.

2.2 Non-Functional Requirements

- smooth gameplay at 60 FPS
- clear and modern dark-themed user interface
- maintainable and modular architecture
- readable and well-commented code suitable for teaching.

2.3 System Requirements and Constraints

Hardware

- Desktop or laptop computer with a modern CPU
- At least 4 GB RAM
- Standard keyboard and mouse or trackpad.

Software

- Python 3.12.10 runtime
- Pygame library installed
- Operating system supported by Pygame (tested on Windows)
- Code editor used: Sublime

Constraints

- single-player, offline mode
- local JSON file used instead of a database server
- fixed window size and resolution controlled by Pygame
- timing and shuffle speed constrained by the chosen frame rate.

2.4 Glossary of Key Terms

FPS Frames per second, the rate at which the game screen is redrawn.

JSON

JavaScript Object Notation, used here as a lightweight data storage format.

Stake

The effective amount at risk for a round, computed from bet and turbo multiplier.

Turbo

A multiplier applied to the base bet to increase potential gain and risk.

State Machine

A model in which the game is always in one of a finite set of states and transitions between them.

Round

A single cycle of bet, card shuffle, selection and result.

3 DEVELOPMENT ENVIRONMENT AND LIBRARIES

3.1 Environment

Python 3.x, Pygame, VS Code, tested on Windows.

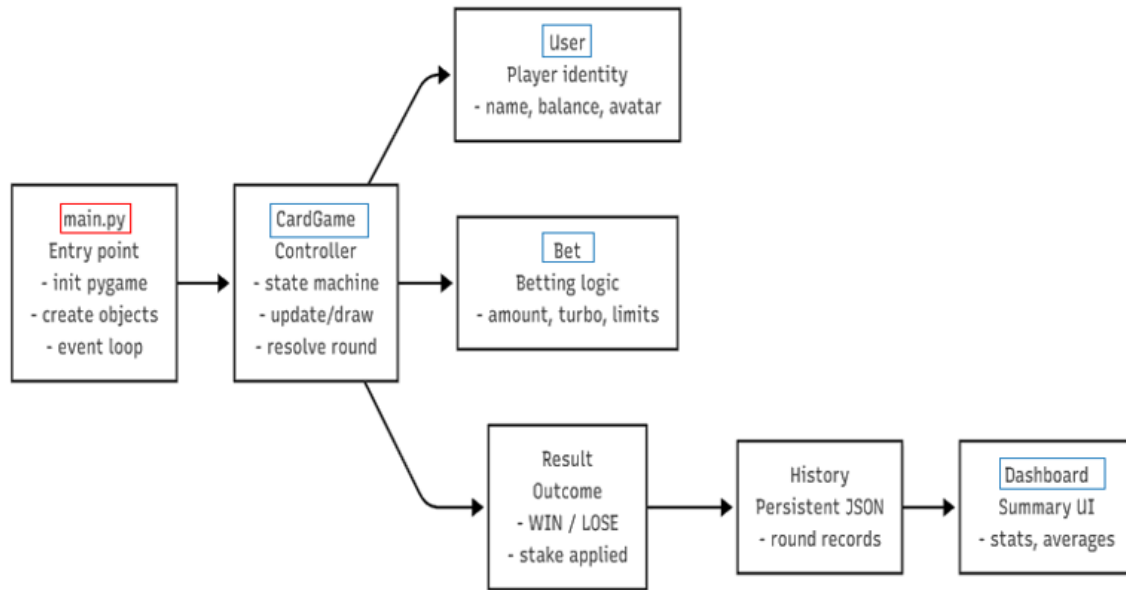
3.2 Libraries Used

```
import pygame
import random
import time
import os
import json
```

4 HIGH-LEVEL DESIGN

4.1 Project Structure and Modules Overview

Project Structure



Modules Overview

- `user.py` — player identity and balance;
- `bet.py` — bet rules and stake computation;
- `game.py` — state machine, animations, core game logic;
- `dashboard.py` — summary view and statistics;
- `main.py` — entry point and Pygame loop.

4.2 UML Class Diagram

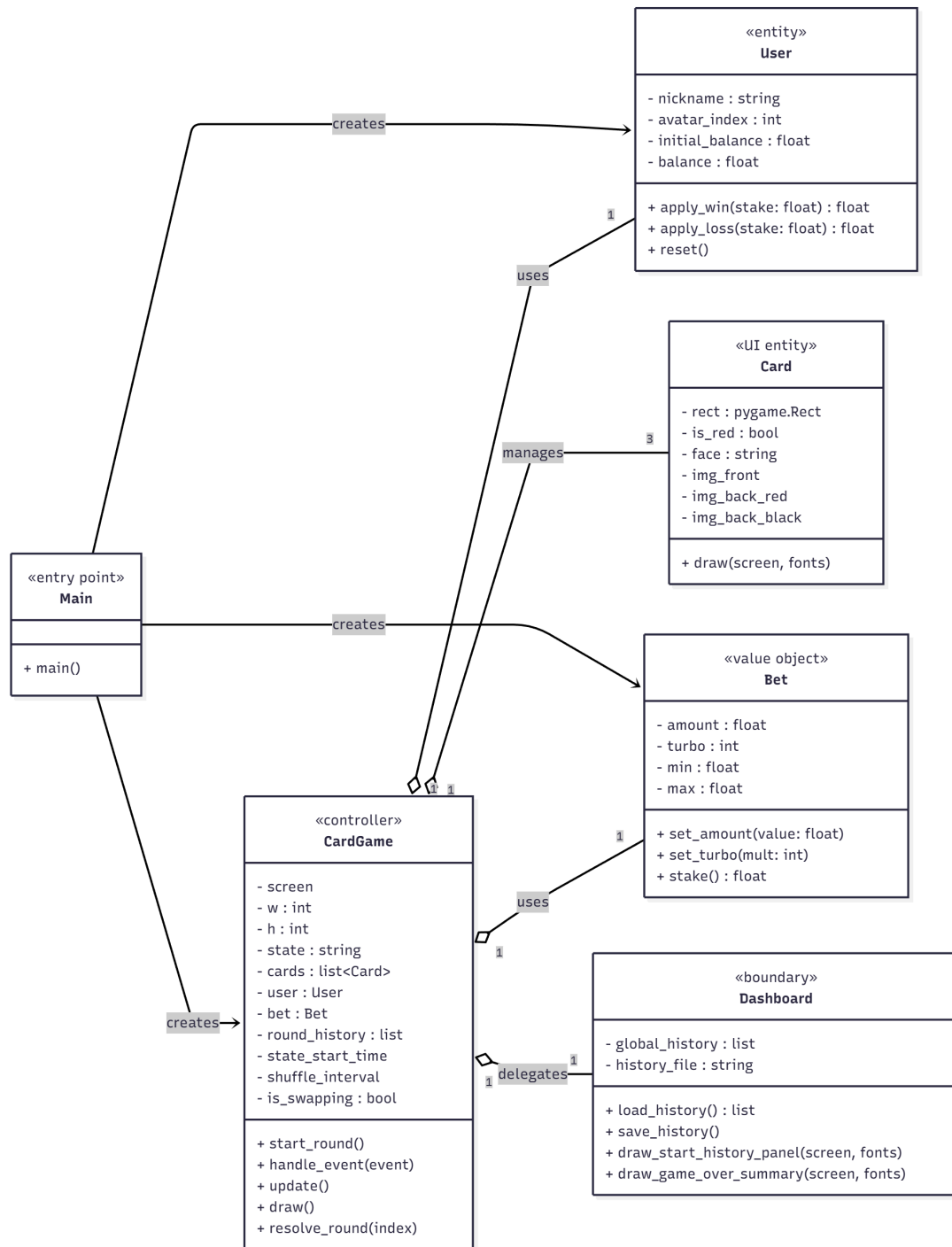


Figure 1: UML Class Diagram.

4.3 ER Diagram

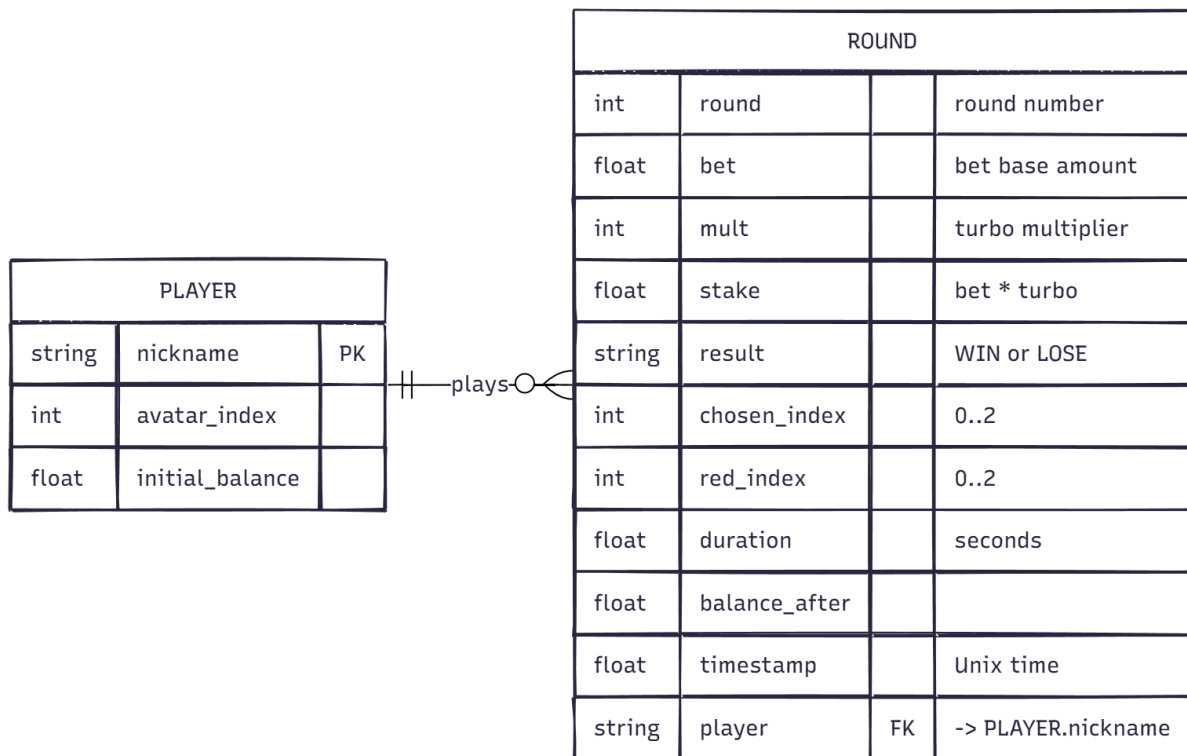


Figure 2: Entity–Relationship Diagram for PLAYER and ROUND.

4.4 SDLC Gantt Chart

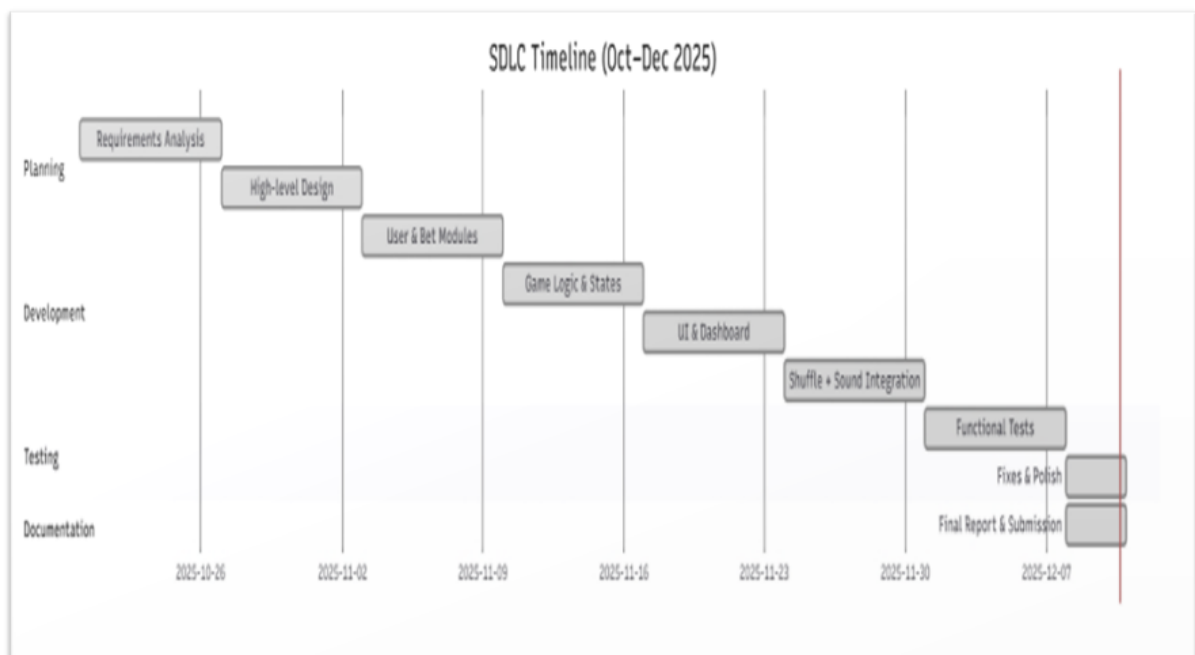


Figure 3: SDLC Gantt Chart (Design, Implementation, Testing, Documentation).

4.5 Sequence Diagram

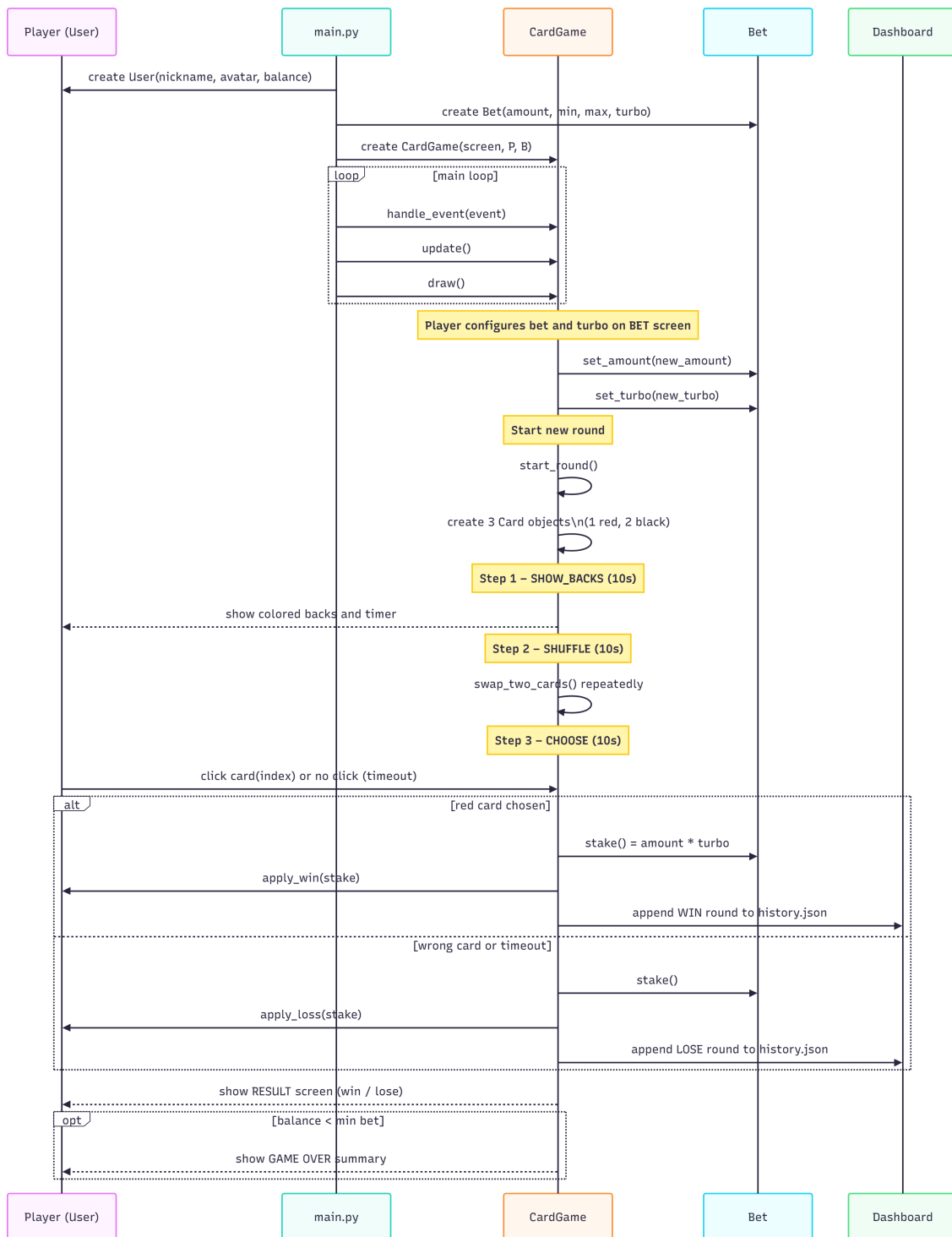


Figure 4: Sequence Diagram: a complete round.

5 OBJECT-ORIENTED ARCHITECTURE

5.1 Encapsulation and Composition

- **User** encapsulates identity, avatar and balance management.
- **Bet** encapsulates bet configuration and stake calculation.
- **Card** encapsulates graphical representation and position.
- **CardGame** composes all of the above and controls the state machine.

5.2 State Machine Logic

```
# ===== GAME STATES =====  
STATE_START = "START_SCREEN"  
STATE_MENU = "MENU"  
STATE_BET = "BET_SETUP"  
STATE_SHOW_BACKS = "SHOW_BACKS"  
STATE_SHUFFLE = "SHUFFLE"  
STATE_CHOOSE = "CHOOSE"  
STATE_RESULT = "RESULT"  
STATE_GAME_OVER = "GAME_OVER"  
STATE_PAUSE = "PAUSE"
```

6 CORE CLASS IMPLEMENTATIONS ---

6.1 User Class

```
# user.py  
  
class User:  
    """  
    Represents a player profile:  
    - nickname  
    - avatar index  
    - balance (capital)  
    """  
  
    def __init__(self, nickname: str = "", avatar_index: int = 0, initial_balance: int = 30):  
        self.nickname = nickname  
        self.avatar_index = avatar_index  
  
        self.initial_balance = initial_balance  
        self.balance = initial_balance
```

6.2 Bet Class

```
class Bet:
    """
    Represents the betting configuration:
    - min / max bet
    - current bet amount
    - turbo multiplier (X1, X2 or X3)
    """
    def __init__(self, min_amount: int = 10, max_amount: int = 100, amount: int = 10, turbo: int = 1):
        self.min = min_amount
        self.max = max_amount
        self.amount = amount
        self.turbo = turbo
```

6.3 Card Class

```
# ===== CARD CLASS =====
class Card:
    def __init__(self, rect, is_red: bool, img_front=None,
                 img_back_red=None, img_back_black=None):
        self.rect = rect
        self.is_red = is_red
        self.face = "BACK"

    # Optional images
    self.img_front = img_front
    self.img_back_red = img_back_red
    self.img_back_black = img_back_black
```

6.4 CardGame Class

```
import pygame
import random
import time
import os
import json

import user as us
import bet as bt
import dashboard as db
# ===== CARD GAME CLASS =====
class CardGame:
    def __init__(self, screen, user: us.User, bet: bt.Bet):
        self.screen = screen
        self.w, self.h = screen.get_size()

        # External domain objects
        self.user = user
        self.bet = bet
```

6.5 main function

```
import pygame
import user as us
import bet as bt
import game as gm
import dashboard as db
"""
Initializes the game window and creates all domain objects.
- Modules: Imports user, bet, game, and dashboard.
- Logic:
    - Starts Pygame and sets window size and caption.
    - Creates User and Bet instances.
    - Instantiates CardGame with screen, user, and bet.
Entry point for launching the game loop
"""
def main():
    pygame.init()          # Initialize Pygame and set window title
    pygame.display.set_caption("Rouge gagne, Noir perd")
    screen = pygame.display.set_mode((960, 630))    # Create game window and clock
    clock = pygame.time.Clock()

    # Create domain objects
    player = us.User(nickname="", avatar_index=0, initial_balance=30)    # player's initial capital = $30 (freebet)
    betting = bt.Bet(min_amount=10, max_amount=100, amount=10, turbo=1) # bet range  >= $10 ---- <= $100

    # Create game instance
    card_game = gm.CardGame(screen, player, betting)
```

7 USER INTERFACE

7.1 Main Menu

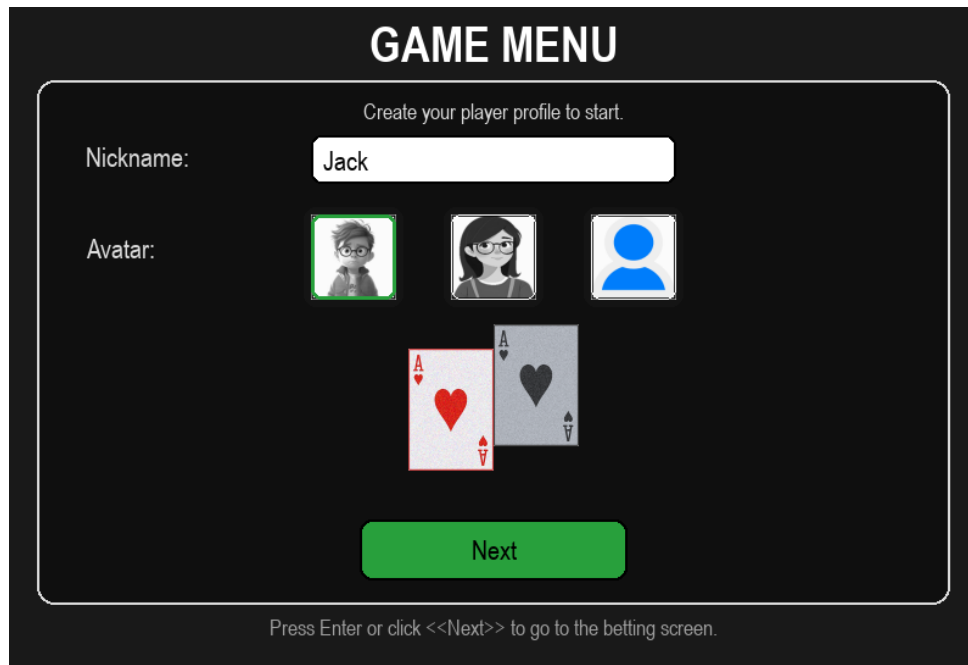


Figure 5: Main menu: nickname and avatar selection.

7.2 Bet Dashboard

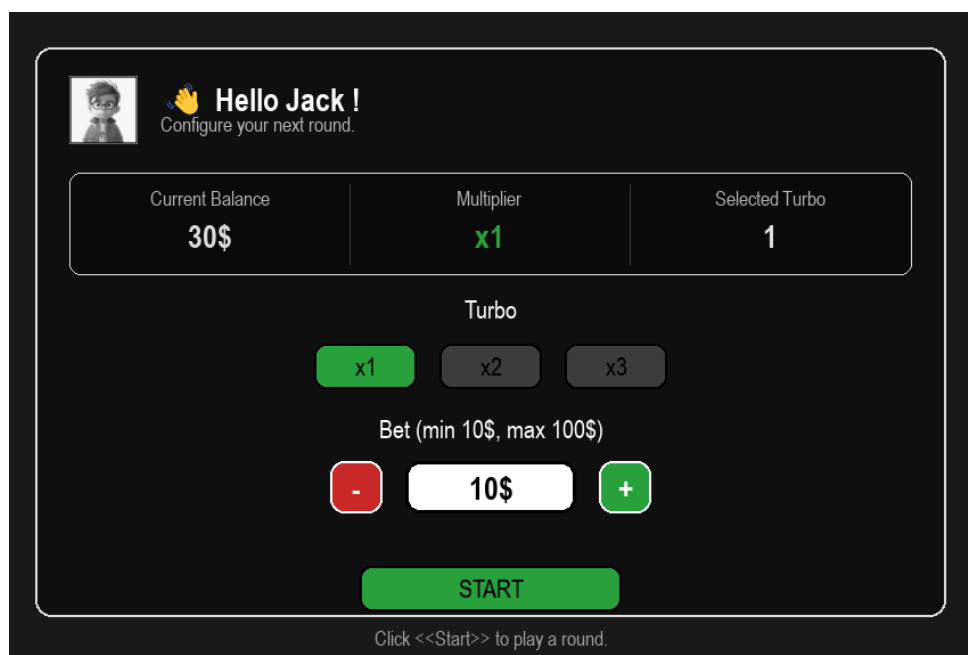


Figure 6: Bet configuration dashboard.

7.3 Step 1: Observe Cards (10s)

 **Jack**

Step 1: watch the red card!

Cards are visible for 10 seconds.

00:06



7.4 Step 2: Cards shuffling (10s)

 **Jack**

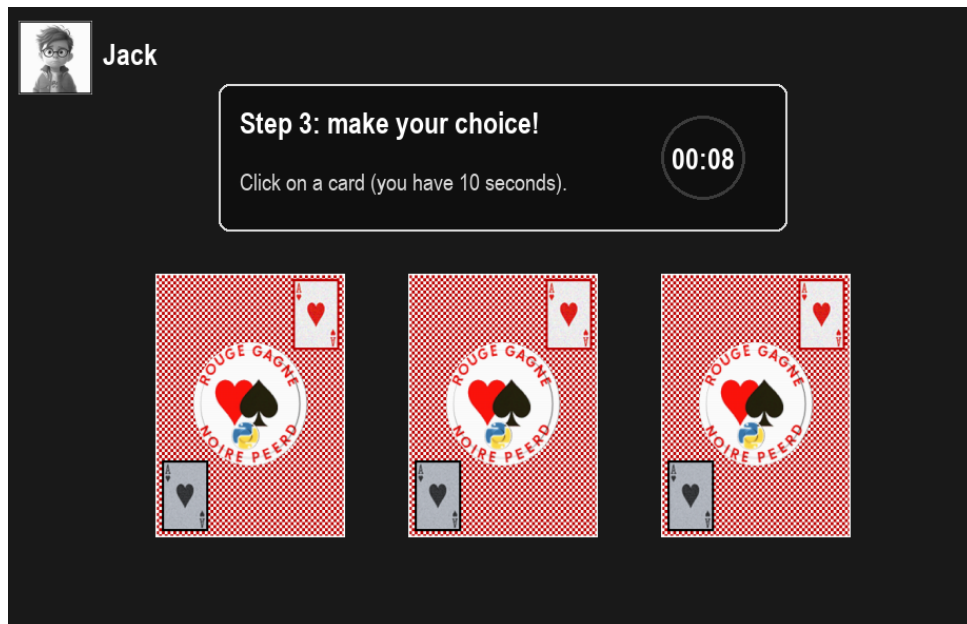
Step 2: card shuffling

Mentally track the position of the red card...

00:07



7.5 Step 3: Card Selection (10s)



7.6 Result and Game Over Screens

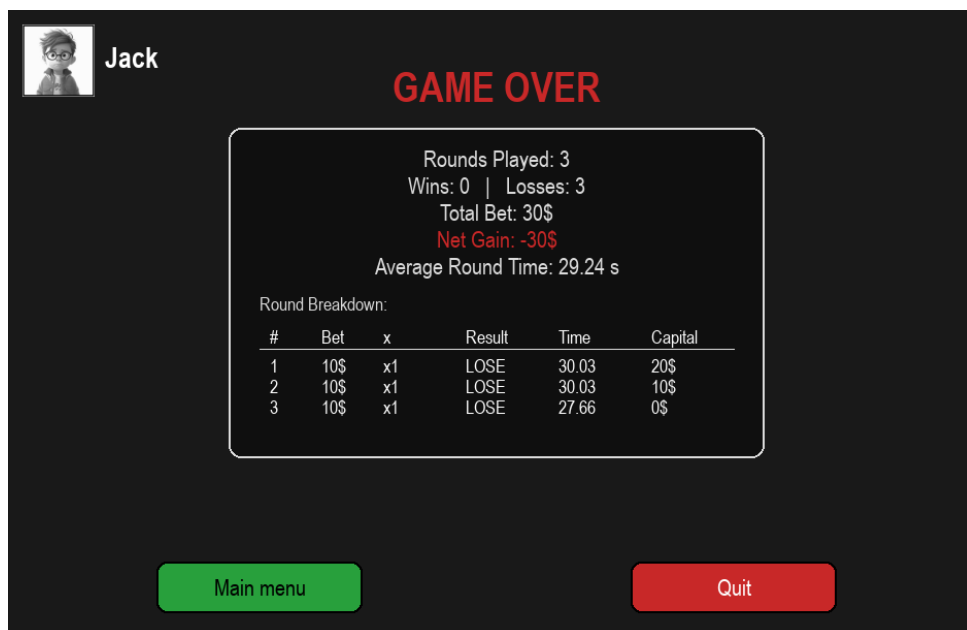


Figure 7: Result and Game Over dashboard.

8 SOFTWARE DEVELOPMENT LIFE CYCLE

The project follows a simplified SDLC with four main phases:

- **Design** — requirements capture, UML and ER diagrams, UI sketches;

- **Implementation** — coding of modules, integration of assets;
- **Testing** — functional and non-functional validation;
- **Documentation** — report, in-code comments and user guidance.

Overall, the flow can be summarised as:



9 RISK ANALYSIS

9.1 Identified Risks

- **Invalid user input** (e.g. empty nickname, invalid bet amount).
- **Performance issues** on very low-end hardware affecting animation smoothness.
- **Data loss** if the JSON history file becomes corrupted or unwritable.
- **Logic bugs** in the shuffle or state machine leading to inconsistent results.

9.2 Mitigation Strategies

- Input validation on nickname length and numeric fields.
- Capping animation complexity and testing at 60 FPS.
- Defensive JSON handling (file existence checks, robust writes).
- Clear state transitions and separation of concerns between modules.

10 TEST PLAN AND VALIDATION

10.1 Functional Testing

Test ID	Feature	Test Description	Expected Result	Status
TC01	Nickname	Less than 3 chars input	Continue button disabled	✔ Pass
TC02	Bet Limit	Bet outside 10–100 range	Auto-adjusted to valid range	✔ Pass
TC03	Turbo	High bet with x3 multiplier	Stake capped by balance	✔ Pass
TC04	Card Display	10s show duration	Transition to shuffle phase	✔ Pass
TC05	Shuffle	Card position swap	Red card index preserved	✔ Pass
TC06	Win Condition	Click correct red card	Balance increases by stake	✔ Pass
TC07	Loss Condition	Click wrong card	Balance decreases by stake	✔ Pass
TC08	Timeout	No click within 10s	Loss applied automatically	✔ Pass
TC09	Game Over	Balance falls below \$10	Game Over screen shown	✔ Pass
TC10	History Persist	After each round	Entry appended to JSON file	✔ Pass

10.2 Non-Functional Testing

Aspect	Test Description	Expected Result
Performance	Run multiple rounds and measure FPS stability	FPS remains close to 60
Usability	Observe new players using the game without explanation	Players understand the three steps
Robustness	Force invalid inputs and boundary values	Errors handled without crash
Persistence	Simulate repeated runs	JSON history file remains consistent and readable

11 SUMMARY

This report presented the *Rouge Gagne, Noir Perd* game from a software engineering perspective. The system combines a clear state-machine-based design, modular Python implementation, Pygame user interface, and persistent JSON storage. The documentation connects diagrams (UML, ER, SDLC, sequence) with the concrete code architecture and test strategy.

12 CONCLUSION AND PERSPECTIVES

The project successfully demonstrates:

- clean object-oriented structure (User, Bet, Card, CardGame, Dashboard);
- a practical application of SDLC concepts;
- an educational example of event-driven programming with Pygame;
- a complete path from requirements to testing.

Future improvements include:

- difficulty modes (faster shuffles, more cards);
- multiplayer or online leaderboard;
- localisation (e.g. English and French);
- porting the game to mobile platforms.

REFERENCES

- Python Software Foundation, *Python Documentation*. Available at: <https://docs.python.org>
- Pygame Community, *Pygame Documentation*. Available at: <https://www.pygame.org/docs>
- ALPRO Course Material, 2025.

A SOURCE CODE LISTINGS

The full source code for the project is provided in the external attached files. For reference, these include:

- `user.py`
- `bet.py`
- `game.py`
- `dashboard.py`
- `main.py`

These files contain the complete implementation of the game, including class structures, state machine logic, UI rendering, and data persistence.

End of file — © 2025 Denos KUME, M1 CORO DASSIP (2025–2026)

Collaborator: Sena FUKABE | Instructor: Mira Rizkallah

Contact: denoskume@yahoo.com

This material is copyrighted. Redistribution, modification, or reuse without explicit permission from the author is prohibited.
